

Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №3

навчальної дисципліни

ПРОГРАМНІ МЕТОДИ ЗАХИСТУ ІНФОРМАЦІЇ

ФКЗЕ. 121ПМЗІ21. 03ЛР

Шифр роботи

Студент Костенко А.О.

Група ПЗ-23-1/9

Перевірів Костенко А.О.

2026 - 2027

Тема: Використання шифру простої перестановки при розв'язанні задачі захисту інформації (прості шифруючі таблиці, шифруючі таблиці з ключовим словом). Програмна реалізація.

Мета: вивчити принципи шифрування методом перестановки; навчитися шифрувати й розшифровувати текст простою шифруючою таблицею, таблицею з ключовим словом і подвійною перестановкою; оцінити стійкість цих шифрів і реалізувати їх мовою C++.

Обладнання: ПК з компілятором або інтерпретатором мови програмування за вибором студента; Середовище розробки (Visual Studio, VS Code або інше).

Варіант: 1.

Хід роботи

1 Отримано індивідуальне завдання — варіант 1: ключове слово CIPHER, текст «THE QUICK BROWN FOX». Реалізацію виконано мовою Go; вихідні файли лежать у каталозі src/, запуск — go run ., тести — go test ./....

2 Підготовка тексту. Перед записом у таблицю з тексту прибирають усе, крім латинських літер: збережені пробіли видали б у шифротексті межі слів. Після нормалізації лишається 16 літер — THEQUICKBROWNFOX.

Далі текст доповнюють літерою X до повного прямокутника. Це не косметика: доповнення вирівнює довжину всіх стовпців, і саме завдяки йому розшифрування зводиться до оберненої перестановки. Без нього той, хто розшифровує, мусив би ще й вгадувати, які стовпці на одну літеру коротші.

```
// pad доповнює текст до size літер.
func pad(text []rune, size int) []rune {
    for len(text) < size {
        text = append(text, padding)
    }
    return text
}
```

3 Проста шифруюча таблиця. Текст записують по рядках, а читають по стовпцях зліва направо. Ширину обрано 4: на 16 літер це рівно 4x4, тож перший приклад показує шифр у чистому вигляді, без жодної літери доповнення.

T	H	E	Q
U	I	C	K
B	R	O	W
N	F	O	X

Шифротекст: TUBNHIRFECOOQKWX
Розшифровано: THEQUICKBROWNFOX

Ключа тут немає взагалі — таємницею є сама ширина таблиці. Це й робить шифр беззахисним: ширину можна просто перебрати, бо вона мусить бути дільником довжини шифротексту.

4 Порядок стовпців за ключовим словом. Стовпці нумерують за алфавітним порядком літер ключа, і в цьому порядку їх потім читають. Сортування стійке, тому однакові літери ключа читаються зліва направо: у CIPHER повторів немає, але в списку завдань є ключі DEFEND і GUARD, де без стійкого сортування порядок стовпців залежав би від реалізації.

```
// keyOrder повертає індекси стовпців у порядку читання: першим іде стовпець
// під найранішою за алфавітом літерою ключа.
func keyOrder(key string) []int {
    letters := normalize(key)

    order := make([]int, len(letters))
    for i := range order {
        order[i] = i
    }
    sort.SliceStable(order, func(a, b int) bool {
        return letters[order[a]] < letters[order[b]]
    })
    return order
}
```

Для ключа CIPHER виходить C=1, E=2, H=3, I=4, P=5, R=6, тобто стовпці читають у порядку C, E, H, I, P, R.

5 Шифрування та дешифрування з ключовим словом. Ширина таблиці тепер дорівнює довжині ключа — шість стовпців на 16 літер дають три рядки, останні дві комірки заповнює X.

C	I	P	H	E	R
1	4	5	3	2	6
T	H	E	Q	U	I
C	K	B	R	O	W
N	F	O	X	X	X

Шифротекст: TCNUOXQRXHKFEBOIWX
Розшифровано: THEQUICKBROWNFOXXX

Дешифрування — та сама таблиця, заповнена у зворотному напрямку: шифротекст розкладають по стовпцях у порядку ключа й читають по рядках.

```
// keywordEncrypt: та сама таблиця, але стовпці читають не зліва направо,  
// а в алфавітному порядку літер ключа.  
func keywordEncrypt(text, key string) string {  
    order := keyOrder(key)  
    table := layout(text, len(order))  
  
    var out strings.Builder  
    for _, col := range order {  
        for row := range table {  
            out.WriteRune(table[row][col])  
        }  
    }  
    return out.String()  
}
```

```

Terminal - lab03

$ go run .
Текст варіанта:      THE QUICK BROWN FOX
Після нормалізації: THEQUICKBROWNFOX (16 літер)
Ключове слово:       CIPHER

=== 1. Проста шифруюча таблиця (ширина 4) ===



|   |   |   |   |
|---|---|---|---|
| T | H | E | Q |
| U | I | C | K |
| B | R | O | W |
| N | F | O | X |



Шифротекст:  TUBNHIRFEC00QKW
Розшифровано: THEQUICKBROWNFOX
Звірка з оригіналом: збігається

=== 2. Таблиця з ключовим словом CIPHER ===



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| C | I | P | H | E | R |
| 1 | 4 | 5 | 3 | 2 | 6 |



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| T | H | E | Q | U | I |
| C | K | B | R | O | W |
| N | F | O | X | X | X |



Шифротекст:  TCNUOXQRXHNKFEB0IWX
Розшифровано: THEQUICKBROWNFOX
Звірка з оригіналом: збігається

```

Рисунок 1 - Вивід програми: проста таблиця та таблиця з ключовим словом

6 Подвійна перестановка. Тут переставляють і рядки, і стовпці, а результат читають по рядках.

Завдання дає один ключ, а подвійна перестановка потребує двох. Другий ключ узято з тексту того самого варіанта — THE. Вибір не довільний: літери T, H,

Е стоять у зворотному алфавітному порядку, тому рядки справді міняються місцями. Слово FOX із того ж тексту для цього не годиться — воно вже впорядковане за алфавітом, і перестановка рядків вийшла б тотожною, а «подвійний» шифр звівся б до звичайного стовпцевого.

```
// doubleEncrypt переставляє спершу рядки за ключем рядків, потім стовпці
// за ключем стовпців, і читає результат по рядках.
func doubleEncrypt(text, rowKey, colKey string) string {
    rowOrder, colOrder := keyOrder(rowKey), keyOrder(colKey)
    letters := pad(normalize(text), len(rowOrder)*len(colOrder))

    return rows(permute(layout(string(letters), len(colOrder)), rowOrder, colOrder))
}
```

Обидва кроки видно окремо. Спершу рядки стають у порядку ключа THE (Е, Н, Т — тобто третій, другий, перший):

T	H	E	Q	U	I
C	K	B	R	O	W
N	F	O	X	X	X

->

N	F	O	X	X	X
C	K	B	R	O	W
T	H	E	Q	U	I

Потім стовпці стають у порядку ключа CIPHER:

N	X	X	F	O	X
C	O	R	K	B	W
T	U	Q	H	E	I

Шифротекст: NXXFOXCKBWTUQHFI
Розшифровано: THEQUICKBROWNFOXXX

```

Terminal - lab03

$ go run .
...
=== 3. Подвійна перестановка (рядки THE, стовпці CIPHER) ===

Вихідна таблиця:
  C I P H E R
  1 4 5 3 2 6



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| T | H | E | Q | U | I |
| C | K | B | R | O | W |
| N | F | O | X | X | X |



Крок 1 – переставлено рядки за ключем THE:
  C I P H E R
  1 4 5 3 2 6



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| N | F | O | X | X | X |
| C | K | B | R | O | W |
| T | H | E | Q | U | I |



Крок 2 – переставлено стовпці за ключем CIPHER:



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| N | X | X | F | O | X |
| C | O | R | K | B | W |
| T | U | Q | H | E | I |



Шифротекст:   NXXFOXCORKBWTUQHEI
Розшифровано: THEQUICKBROWNFOXXX
Звірка з оригіналом: збігається

```

Рисунок 2 - Подвійна перестановка: вихідна таблиця та обидва кроки

7 Результати шифрування тексту варіанта зведено в таблицю.

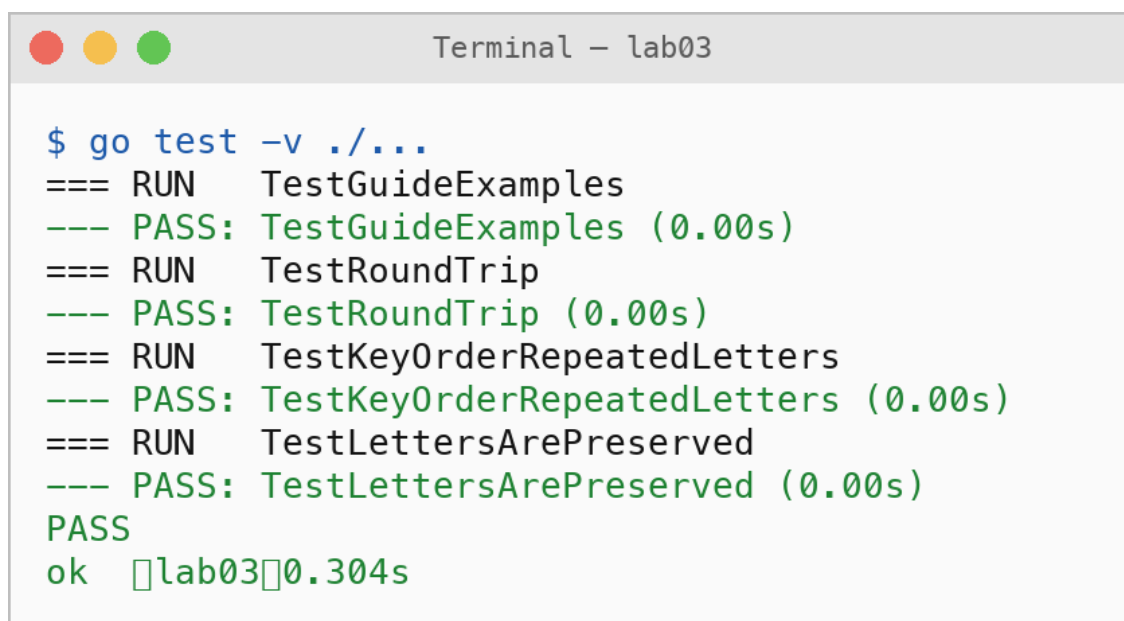
Шифр	Ключ	Шифротекст
Проста таблиця	ширина 4	TUBNHIRFECOOQKWX
Таблиця з ключовим словом	CIPHER	TCNUOXQRXHKFEBOIWX

Шифр	Ключ	Шифротекст
Подвійна перестановка	THE та CIPHER	NXXFOXCORKBWTUQHEI

Дешифрування в усіх трьох випадках повертає доповнений вихідний текст. Літери X, дописані до прямокутника, лишаються в результаті: прибрати їх автоматично не можна, бо X буває й справжньою літерою повідомлення.

8 Перевірка. Правильність реалізації перевірено не на око, а тестами, які відтворюють приклади з методичних вказівок: HELLOWORLD шириною 4, ATTACKATDAWN із ключем CRYPTO і HELLOWORL із ключами KEY та CAT. Go-програма дає ті самі шифротексти, що й наведена в методичці програма на C++.

Ще три тести перевіряють, що дешифрування повертає доповнений оригінал, що ключ із повторюваними літерами дає очікуваний порядок стовпців і що шифрування не змінює складу літер тексту.



```

Terminal - lab03

$ go test -v ./...
=== RUN    TestGuideExamples
--- PASS: TestGuideExamples (0.00s)
=== RUN    TestRoundTrip
--- PASS: TestRoundTrip (0.00s)
=== RUN    TestKeyOrderRepeatedLetters
--- PASS: TestKeyOrderRepeatedLetters (0.00s)
=== RUN    TestLettersArePreserved
--- PASS: TestLettersArePreserved (0.00s)
PASS
ok  lab03 0.304s

```

Рисунок 3 - Результат виконання go test

Відповіді на контрольні питання

Достатній рівень

1. Чому шифри перестановки не піддаються класичному частотному аналізу окремих літер?

Бо частоти літер у шифротексті й у відкритому тексті однакові. Частотний аналіз покаже, що текст англійською, але жодної літери не розкриє: він відповідає на питання «яка це літера», а перестановка ховає не літеру, а її місце.

2. Що дає подвійна перестановка для стійкості порівняно з одинарною?

Після одинарної стовпцевої перестановки літери кожного стовпця лишаються сусідами й у тому ж порядку — криптоаналітику досить правильно скласти цілі стовпці. Друга перестановка руйнує й ці блоки, тож відновлювати доводиться не порядок стовпців, а розташування окремих літер.

3. Як символи-заповнювачі можуть видати довжину ключа криптоаналітику?

Заповнювачі стоять у кінці останнього рядка таблиці, тому після читання по стовпцях вони опиняються в кінцях стовпців — тобто на регулярній відстані один від одного. У шифротексті цієї роботи TCNUOXQRXHKFEBOIWX літери X стоять на позиціях 6, 9 і 18; крок 3 одразу дає число рядків, а $18 : 3$ — шість стовпців, тобто довжину ключа.

4. Запишіть алгоритм розшифрування тексту, зашифрованого таблицею з ключовим словом.

Обчислити порядок стовпців за ключем; поділити довжину шифротексту на довжину ключа й дістати число рядків; розрізати шифротекст на відрізки завдовжки в число рядків; розкласти ці відрізки по стовпцях у порядку ключа; прочитати таблицю по рядках.

5. Як однакові літери в ключовому слові впливають на порядок стовпців і як цю неоднозначність усувають?

Однакові літери самі по собі не кажуть, який зі стовпців читати першим. Усувають це домовленістю: рівні літери впорядковують за позицією в ключі, зліва

направо. У програмі це забезпечує стійке сортування, а окремий тест перевіряє ключ DEFEND.

6. Чому аналіз частот біграм ефективніший проти шифрів перестановки, ніж аналіз частот літер?

Частоти літер перестановка не змінює, тому вони не несуть інформації про ключ. А от сусідство літер вона руйнує, і його можна відновлювати: перебираючи розташування стовпців, аналітик обирає той варіант, де пари сусідніх літер найбільше схожі на звичайні для мови — TH, HE, IN.

7. Як визначити ймовірну довжину ключа, маючи лише шифротекст?

Довжина шифротексту кратна довжині ключа, тому кандидатами є дільники цієї довжини. Кожен кандидат перевіряють: складають таблицю відповідної ширини й дивляться, чи вдається знайти порядок стовпців із правдоподібною статистикою біграм. Додаткову підказку дають заповнювачі з питання 3.

8. Чим магічний квадрат як спосіб перестановки відрізняється від шифруючої таблиці з ключем?

У магічному квадраті порядок задають числа самого квадрата, а не окреме слово: ключем є квадрат. Це незручніше на практиці й слабше як ключ — магічних квадратів кожного розміру скінченна й невелика кількість, тоді як ключових слів стільки, скільки слів у мові.

Висновок: Виконано програмну реалізацію трьох шифрів перестановки — простої шифруючої таблиці, таблиці з ключовим словом і подвійної перестановки — з дешифруванням для кожного. Правильність підтверджено тестами, які відтворюють контрольні приклади з методичних вказівок.

Головне, що показала робота: шифр перестановки приховує лише порядок літер і не чіпає їхнього складу. Через це він не піддається частотному аналізу окремих літер, але й стійкості не дає — уся інформація про текст лишається в шифротексті, змінено тільки її розташування. Довжина шифротексту видає кандидатів на довжину ключа, заповнювачі підказують розмір таблиці, а статистика біграм дозволяє відновити порядок стовпців перебором. Навіть

подвійна перестановка з ключами у вісім і десять символів дає близько 2^{37} варіантів — величину, яку сучасна машина долає за години.

Самостійної цінності такі шифри сьогодні не мають, але як складник вони працюють: у поєднанні із заміною перестановка забезпечує розсіювання, і саме на чергуванні цих двох операцій побудовані DES і AES.